Programación de Sistemas Distribuidos

Curso 2026/2027

Práctica 2

Diseño e implementación de un sistema de juegos on-line utilizando Servicios Web

Esta práctica consiste en el diseño e implementación de un sistema de juegos on-line. Concretamente, se pretende desarrollar tanto la parte cliente, como el servidor, para permitir partidas remotas del juego conecta-3x3 utilizando Web Services. En general, el juego a implementar será el mismo que el utilizado en la práctica 1, con la excepción de que la comunicación entre clientes y el servidor se llevará a cabo mediante Web Services (utilizando gSOAP con el lenguaje C) en lugar de sockets.

Las normas del juego serán las mismas que las utilizadas en la práctica 1.

En la comunicación entre cliente y servidor podemos definir tres elementos que deberán transmitirse entre estas partes: un código, un mensaje y un tablero. A diferencia de la práctica 1, donde información se enviaba por separado, ahora podremos enviar todos los datos en una única estructura. Los valores de los códigos y la definición de las estructuras se proporcionan en el fichero `conecta3x3.h`. El siguiente cuadro muestra una parte de este fichero.

```
//gsoap conecta3x3ns service name: conecta3x3
//gsoap conecta3x3ns service style: rpc
//gsoap conecta3x3ns service location: http//localhost:10000
//gsoap conecta3x3ns service encoding: encoded
//gsoap conecta3x3ns service namespace: urn:conecta3x3ns


/** A player is already registered with the same name */
#define ERROR_NAME_REPEATED -1

...

/** Board width (in number of cells) */
#define BOARD_WIDTH 7

/** Board height (in number of cells) */
#define BOARD_HEIGHT 6

/** Players */
typedef enum players {player1, player2, player3} conecta3x3ns__tPlayer;

/** Dynamic array of chars */
typedef char *xsd__string;

/** Message used to send the player's name and messages sent from server */
typedef struct tMessage{
        int __size;
        xsd__string msg;
}conecta3x3ns__tMessage;

/** Response from the server */
typedef struct tBlock{
        unsigned int code;
        conecta3x3ns__tMessage msgStruct;
        int __size;
        xsd__string board;
}conecta3x3ns__tBlock;

int conecta3x3ns__register   (conecta3x3ns__tMessage playerName, int* code);
```

El turno actual del jugador al que le toca introducir ficha en el tablero se representa con el enumerado `conecta3x3ns__tPlayer`. En este caso, define tres valores, uno para cada jugador de la partida: `player1`, `player2` y `player3`.

Los mensajes de texto que se envíen a los clientes, así como los nombres de los jugadores, se podrán representar mediante el tipo `xsd__string`. Sin embargo, para

enviar las cadenas de texto entre cliente y servidor, será necesario utilizar la estructura `conecta3x3ns__tMessage`, la cual contiene el mensaje a enviar (`msg`) y la longitud del mismo, en número de caracteres (`size`).

La estructura `conecta3x3ns__tBlock` contiene la información necesaria para comunicar el estado de la partida a un cliente. En particular, esta estructura contiene un código (`code`) para indicar el resultado de la acción realizada por el jugador, o su turno, un mensaje (`msgStruct`) con información textual sobre la partida, y un tablero (`board`) junto con su tamaño (`size`) en número de caracteres.

Seguidamente se muestran los servicios ofrecidos. El primero, llamado **`conecta3x3ns__register`**, registra un usuario. Este servicio recibe como parámetro de entrada el nombre del jugador (`playerName`), y como parámetro de salida se devuelve el código con el resultado del registro (`code`), el cual puede representar el ID de la partida – número mayor o igual que 0, si el registro se ha llevado a cabo correctamente – o un código de error indicando el motivo que ha impedido realizar el registro (número menor que 0). En el caso de que no haya hueco para este nuevo jugador, se devolverá el código `ERROR_SERVER_FULL`. Si el nombre del jugador ya existe **en la partida donde se ha encontrado un hueco**, se devuelve el código `ERROR_PLAYER_REPEATED`. En este caso, aunque existan más huecos en otras partidas, se devolverá el código de error y se finalizará el proceso de registro para este jugador.

En esta práctica **no vamos a considerar** el nombre del usuario como identificador unívoco, es decir, pueden existir dos (o más) usuarios con el mismo nombre, siempre que sea en partidas distintas.

El segundo servicio, llamado **`conecta3x3ns__getStatus`**, obtiene el estado de la partida. Se utilizarán los mismos códigos empleados en la práctica 1 para indicar el estado de la misma, esto es, `TURN_MOVE` y `TURN_WAIT` para indicar si al jugador le toca insertar ficha o esperar, respectivamente, y `GAMEOVER_WIN`, `GAMEOVER_DRAW`, y `GAMEOVER_LOSE`, para indicar si el jugador ha ganado, empatado o perdido, respectivamente.

Para completar la aplicación pedida, es necesario desarrollar un nuevo servicio (`insertChip`) que permita a un jugador introducir una ficha en el tablero.

Estos dos últimos servicios deben definir, como parámetros de entrada, el nombre del jugador y el identificador de la partida, entre otros. Además, deberán comprobar si `playerName` está registrado en la partida indicada. En caso de no estarlo, se devolverá el código `ERROR_PLAYER_NOT_FOUND` y el cliente finalizará su ejecución.

El fichero `game.h` contiene las cabeceras de los subprogramas encargados de la lógica del juego. Además, este fichero contiene una descripción detallada de los parámetros de entrada y salida de cada subprograma, los cuales podrán ser invocados desde el programa servidor. Seguidamente, se describen a continuación:

```
void initBoard (tBoard board);
```

Inicializa un tablero. Debe ser invocado antes de comenzar una partida.

```
tMove checkMove (tBoard board, unsigned int column);
```

Comprueba el movimiento de un jugador. El movimiento será el número [0-4] de la columna (`column`) donde el jugador intenta insertar la ficha.

```
void insertChip (tBoard board, tPlayer player, unsigned int column);
```

Introduce una ficha en la columna indicada por el jugador, actualizando el tablero.

```
int checkWinner (tBoard board, tPlayer player);
```

Comprueba si el jugador actual es el ganador. Este subprograma deberá invocarse después de haber realizado el movimiento.

```
int isBoardFull (tBoard board);
```

Comprueba si el tablero está lleno. Esto es, ya no es posible realizar más movimientos.

La parte cliente no necesita manipular (ni debe) el tablero. Una vez reciba la información del servidor, podrá utilizar el siguiente subprograma (ver fichero `utils.h`) para mostrarlo por pantalla:

```
void printBoard (xsd__string board, xsd__string message);
```

Imprime por pantalla el estado de la partida

```
void allocClearMessage (struct soap *soap, conecta3x3ns__tMessage* msg);
```

Reserva memoria para un mensaje (estructura `conecta3x3ns__tMessage`). El primer parámetro indica el contexto *soap* asociado a la reserva de memoria. Además, el mensaje se borra para eliminar cualquier residuo en esa parte de la memoria.

```
void allocClearBlock (struct soap *soap, conecta3x3ns__tBlock* block);
```

Reserva memoria para un bloque que contiene el estado del juego (estructura `conecta3x3ns__tBlock`). El primer parámetro indica el contexto *soap* asociado a la reserva de memoria.

Generación del stub del cliente y el esqueleto del servidor

Para generar el *stub*, el *esqueleto*, los ficheros de cabecera y los ficheros encargados de realizar el *marshalling* y *unmarshalling* es necesario utilizar la herramienta `soapcpp2`.

```
$> soapcpp2 -b -c conecta3x3.h
```

Para compilar la práctica se proporciona un fichero `Makefile` que automatiza esta tarea.

Implementación del cliente

La aplicación cliente deberá establecer comunicación con el servidor para realizar las siguientes acciones:

- Registrar al jugador en el sistema.
- Solicitar el estado de la partida.
- Realizar un movimiento insertando una ficha en el tablero.

Dado que esta práctica se implementará utilizando Servicios Web, podremos enviar estructuras al invocar los servicios, lo cual resulta más cómodo que enviar a través de sockets cada dato de forma individual.

Una posible estructura para la aplicación cliente puede ser la siguiente:

```
Mientras (no acabe el juego)
      Pedir estado del juego
      Imprimir estado del juego
      Mientras (jugador tenga el turno)
            Leer movimiento
            Insertar ficha
            Si (fin de juego)
                  Fin de juego
            Si (no es fin de juego)
                  Imprimir tablero
```

Implementación del servidor

En esta parte se pide atender las peticiones en el servidor de forma concurrente. Para ello, cada petición realizada por los jugadores y recibida por el servidor, conllevará la creación de un *thread* que procese esta petición como corresponda.

Se puede tomar, como punto de partida, la implementación del servidor *multi-thread* explicada en clase donde se implementa una calculadora básica.

El servidor almacena un array de estructuras `tGame` para gestionar las partidas:

```
tGame games[MAX_GAMES];
```

Dado que cada partida tendrá tres jugadores, y se podrán gestionar varias partidas a la vez, este array se definirá como una variable global, permitiendo el acceso de varios *threads* ejecutados en la aplicación. Estos *threads* se crearán cuando un cliente solicite un servicio del servidor, y ejecutarán la función indicada como parámetro (por ejemplo, **`processRequest`**). Este subprograma se encargará de invocar el servicio solicitado por el jugador, de forma que el acceso al array `games` se realizará de forma concurrente.

Con el fin de proporcionar un acceso controlado a este array, será necesario utilizar tanto *mutex* como variables de condición. El primer paso será diseñar cómo se realizarán los accesos a la/s región/es crítica/s. Seguidamente, definiremos en la estructura `tGame` tanto los *mutex* como las variables de condición necesarias.

Es muy importante realizar de forma adecuada los accesos a este array. Por ejemplo, el primer caso en el que hay que proteger el acceso concurrente de los *threads* es en el servicio para registrar a los jugadores. Si no establecemos un acceso exclusivo a determinados campos como, por ejemplo, los nombres o el estado de la partida, es posible que el sistema llegue a estar en un estado inconsistente.

Una partida puede tener varios estados (`tGameStatus`): `gameEmpty`, `gameWaitingPlayer`, o `gameReady`, que representan una partida vacía, una partida donde se han registrado uno o dos jugadores, y una partida con los tres jugadores registrados, respectivamente. Cuando dos jugadores intenten registrarse de forma simultánea, el servicio deberá hacer uso de los mecanismos descritos para acceder a la sección crítica de forma apropiada.

Una posible implementación podría consistir en definir un *mutex* que proteja el array `games` en todos los accesos, de forma que únicamente un *thread* tendrá acceso - a la vez - al array. Sin embargo, esto no sería una solución apropiada, ya que dejamos bloqueados a los procesos, aunque no necesitemos proteger campos como los nombres de los jugadores o el estado de la partida, limitando de forma considerable la explotación del paralelismo.

Cuando se registran los tres jugadores, dará comienzo la partida. El jugador que empieza la misma se calcula aleatoriamente. Para simplificar la implementación del servidor, es posible que un jugador registrado comience la partida, aunque no se haya registrado el resto, siempre y cuando tenga el turno para jugar. Así, si un jugador se registra correctamente, y tiene turno para insertar una ficha, podrá realizar el primer movimiento, quedando seguidamente bloqueado hasta que los otros dos jugadores realicen un movimiento correcto.

Uno de los puntos más sensibles del servidor para establecer la sincronización de los procesos está en el servicio **`conecta3x3ns__getStatus`**. En particular, este servicio deberá dejar bloqueado a un jugador cuando no sea su turno. Para ello, utilizaremos una variable de condición, de forma que si no es el turno del jugador que ha invocado el servicio, éste permanecerá bloqueado al realizar una operación `pthread_cond_wait`. Para conocer si es el turno del jugador que invoca el servicio, deberemos acceder a los nombres de los jugadores (campos `player1Name`, `player2Name`, y `player3Name`) y al campo `currentPlayer` de la partida.

Es importante tener en cuenta cómo desbloqueamos a los procesos. Por ejemplo, si hay dos procesos bloqueados esperando su turno, y sólo desbloqueamos a uno de ellos cuando un jugador ya ha introducido ficha y pasa el turno al siguiente, es posible que el proceso desbloqueado no coincida con el jugador que tiene el turno actual, lo cual llevará a una situación de interbloqueo. Para evitar esto, podemos hacer uso de la llamada a `pthread_cond_broadcast`. De esta forma, desbloquearemos a todos los procesos bloqueados, permitiendo continuar la ejecución al proceso que tenga el turno, y dejando bloqueados al resto. Por ejemplo, así:

```
mientras ((playerName == nameJ1 && no turno J1) ||
          (playerName == nameJ2 && no turno J2) ||
          (playerName == nameJ3 && no turno J3))
            pthread_cond_wait
```

Otro punto sensible es la terminación de la partida. Para realizarla correctamente, se deberá contabilizar los jugadores que permanezcan activos en la misma de forma que

el último jugador se encargue de inicializar los campos de la estructura `tGame` correspondiente para una partida nueva.

Cuando un jugador inserta una ficha en el tablero, es posible que este movimiento provoque el fin de la partida, bien porque haya ganado, bien porque se haya llenado el tablero de fichas y no exista ganador (empate). En este caso, es posible devolver el código de fin de partida en la estructura `conecta3x3ns__tBlock`, además del tablero. De esta forma, este jugador no tendrá que volver a invocar el servicio **`conecta3x3ns__getStatus`** para comprobar si la partida ha finalizado.

Para facilitar la copia de parámetros a la estructura `conecta3x3ns__tBlock`, se proporciona el subprograma **`copyGameStatusStructure`** en el servidor:

```
void copyGameStatusStructure (conecta3x3ns__tBlock* status,
                              char* message,
                              char* board,
                              int newCode);
```

donde `status` es la estructura donde se copiarán los datos, `message` es el mensaje con información sobre el estado de la partida, `board` es el tablero, y `newCode` el código que indica el estado de la partida.

Queda a responsabilidad del alumno el diseño e implementación del acceso a las zonas de memoria compartida (sección crítica). Es decir, se deja total libertad para utilizar estas estructuras como se considere oportuno, siempre y cuando se cumplan con los requisitos detallados en el enunciado.

Ficheros a entregar

Esta práctica puede desarrollarse utilizando un código fuente inicial, el cual contiene parte de la implementación y los tipos de datos utilizados. Este código fuente se encuentra en el fichero `PSD_Prac2_WS.zip`.

Para desarrollar este apartado se deberán modificar, **únicamente,** los ficheros `server.c` y `client.c`.

Es importante matizar que el fichero .zip entregado debe contener los ficheros necesarios para realizar la compilación, tanto del cliente, como del servidor. En caso de que cualquiera de las partes entregadas no compile, se tendrá en cuenta la penalización correspondiente.

Entrega de la práctica

Para entregar esta práctica se habilitará un entregador en la página de la asignatura del Campus Virtual. La fecha límite para entregarla será el **día 4 de Noviembre de 2026 a las 15:55.**

No se permitirá la entrega de prácticas fuera del plazo establecido.

La entrega se realizará mediante un **único fichero con extensión .zip**, el cual deberá incluir los ficheros necesarios para ejecutar y compilar la aplicación pedida, además del fichero `nombres.txt`, que contendrá el nombre y apellidos de los integrantes del grupo.

Se van a perseguir las copias y plagios de prácticas, aplicando con rigor la normativa vigente.